Non-Functional Requirements: The Backbone o Great Software - Part 2

BYTEBYTEGO
FEB 20, 2025 • PAID

Non-functional requirements (NFRs) are as critical as functional requirements beca they define a system's qualities and operational parameters.

Functional requirements specify what a software product should do (for example, “users must be able to log in”). However, non-functional requirements define how v it must accomplish these tasks under real-world conditions (for example, “the login process should respond within two seconds under peak load” or “all user credential must be encrypted and stored securely”).

Together, functional and non-functional requirements create a foundation for build great software systems.

In Part 1 of this topic, we also looked at trade-offs in non-functional requirements a the architectural impact of these requirements. Some key learning points from Part were as follows:

- Non-functional requirements determine a system's performance in areas such a scalability, response time, performance, security, etc., ensuring it meets quality standards and user expectations beyond mere functionality.
- Functional requirements focus on what the system should do, while non-functional requirements define how well it operates under real-world conditior
- Optimizing one NFR (for example, performance) can negatively impact others (example, maintainability), requiring a balanced approach based on project goal

and constraints.

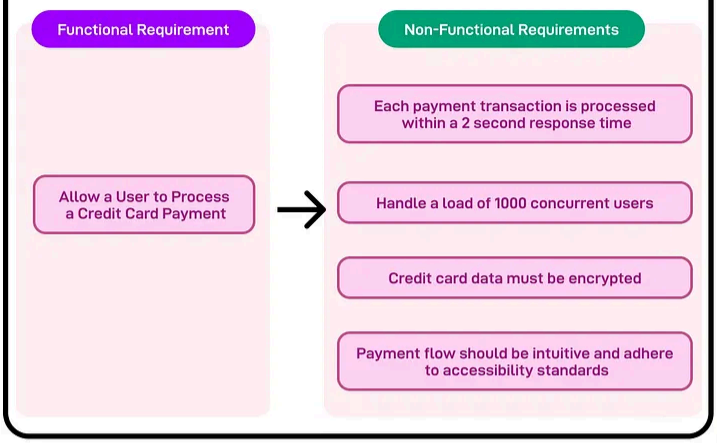
- Non-functional needs (like scalability and availability) heavily influence architecture choice. For example, choosing microservices for high availability c plugin-based designs for modifiability.

In this article (Part 2), we'll go further and look at some of the most important NFR that should be considered while building systems.

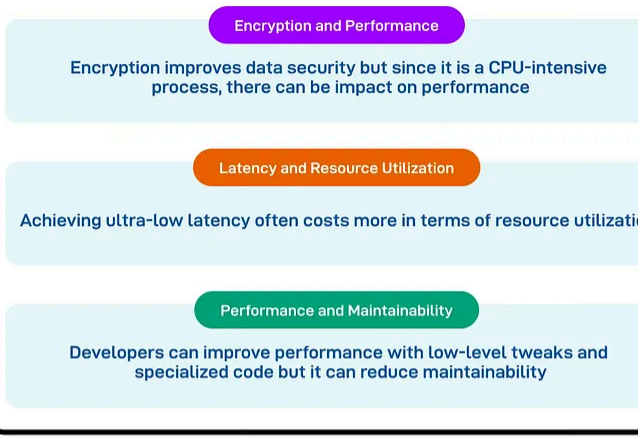
A Cheatsheet on Non-Functional Requirements

ByteByteG

1. Functional Vs Non-Functional Requirements



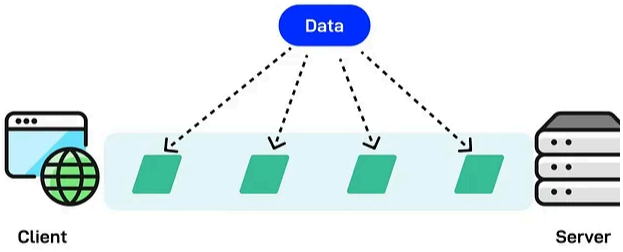
2. Trade-Off Examples



3. Latency or Response Time

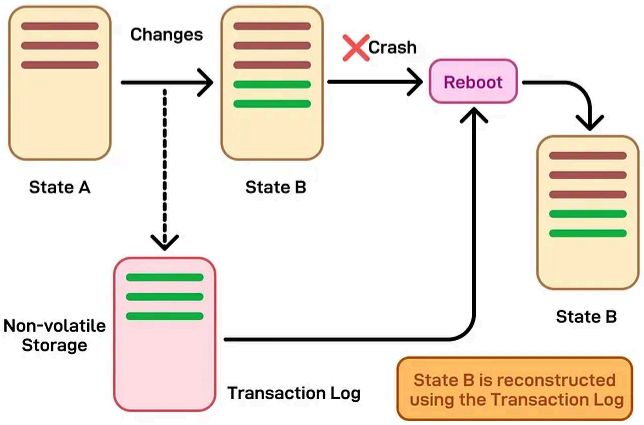
Total Latency (Round Trip Time): 400 ms + 500 ms = 900 ms

4. Throughput



Quantity of Data That Can be Sent and Received Within a Unit of Time

5. Durability



6. Availability

Availability %	Downtime Per Year	Commonly Referred
99%	3.65 days	Two nines
99.9%	8.76 hours	Three nines
99.99%	52.56 minutes	Four nines
99.999%	5.26 minutes	Five nines
99.9999%	31.5 seconds	Six nines

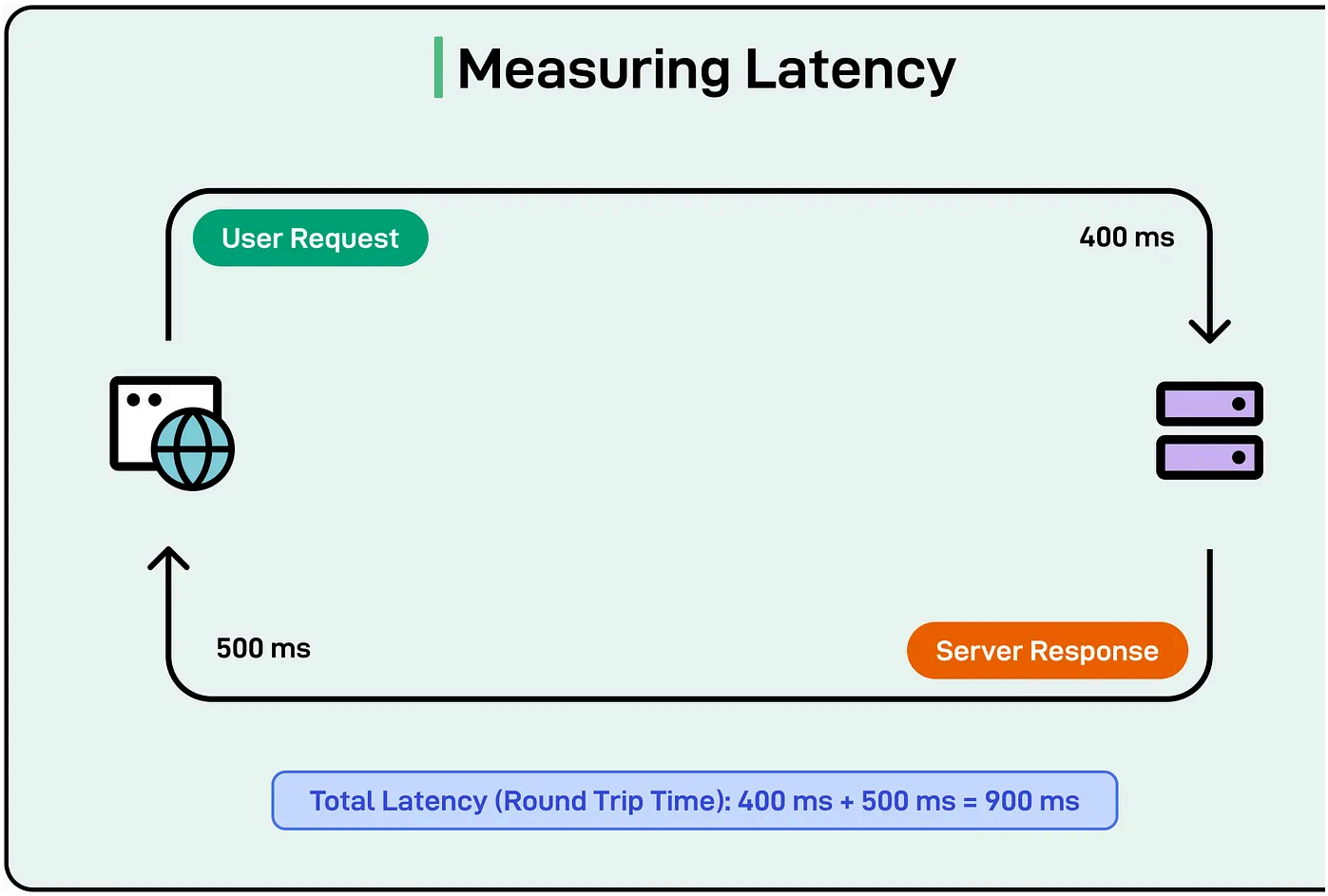
Key NFRs to Consider

Some key non-functional requirements that should be considered while designing a application are as follows:

Response Time/Latency

Response time (often referred to as latency) is the interval between a user action (lik clicking a button or making an API request) and the system's corresponding respon (such as rendering the next page or returning data).

See the diagram below:



In simpler terms, it's how long the user or client has to wait for the system to proce request and deliver a result.

Some key metrics to measure latency are as follows:

- **Average Response Time (Mean):** The average time taken for the system to resp to a request, calculated by summing all response times and dividing by the number of requests. This is useful for getting a general sense of system performance but can be misleading if there are extreme outliers.
- **95th or 99th Percentile Response Times:** Measures the response time that 95% 99% of requests are completed within, highlighting worst-case performance scenarios. It helps identify outliers and peak load conditions that impact a sma but critical subset of users.
- **Time to First Byte (TTFB):** The time between a client's request and when it receives the first byte of the response from the server. This metric is critical for web applications as it reflects server processing time and network latency, impacting perceived speed and SEO rankings.

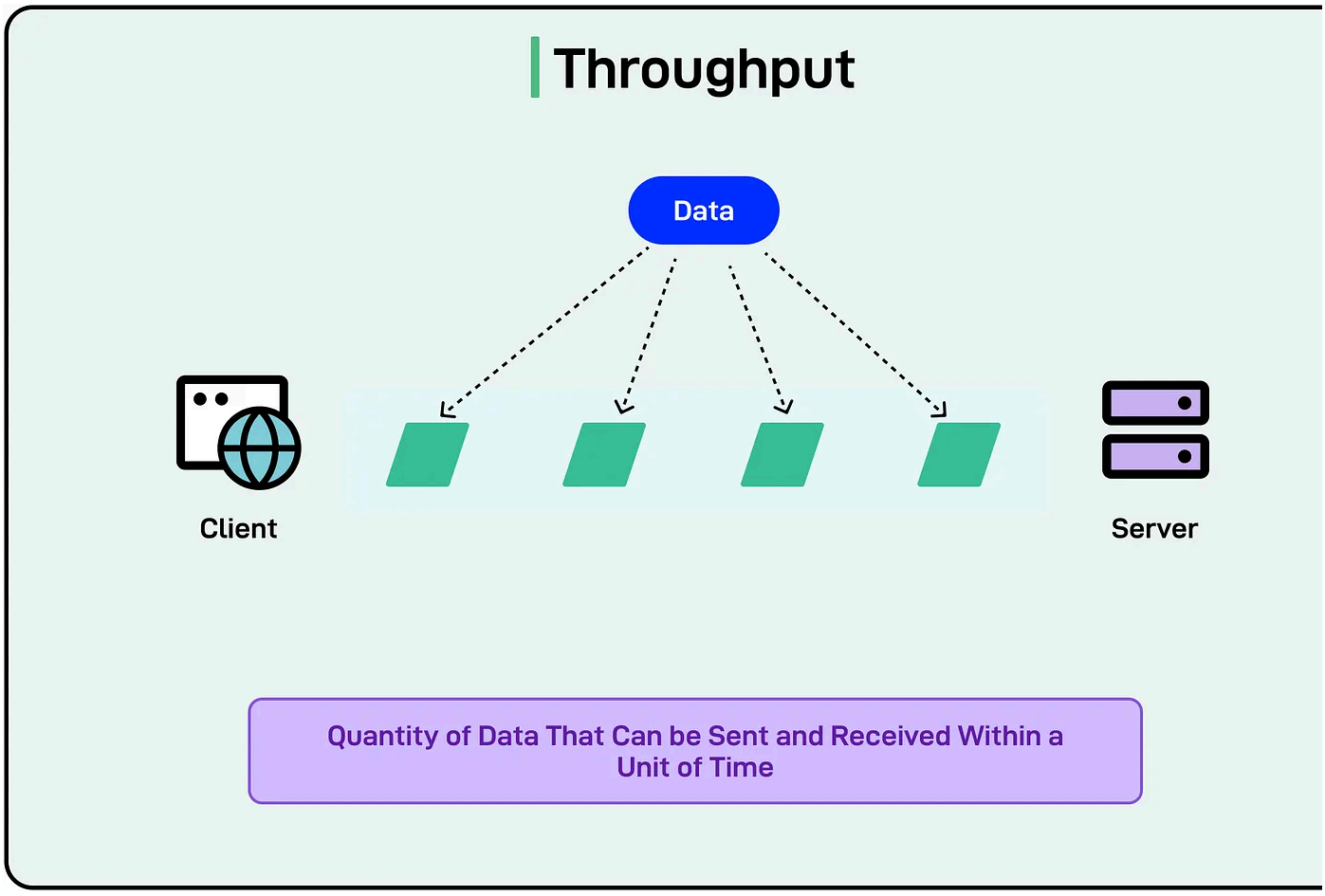
The factors that have a major impact on the latency of a system are as follows:

- **Network Overhead:** Busy networks introduce packet delays and possible retransmissions due to dropped packets.
- **Physical Distance:** Data traveling across large geographical distances encounte higher round-trip times.
- **Protocol Overhead:** Encryption or secure protocols like TLS add extra steps to handshake and data encoding/decoding.
- **Application Logic:** Complex calculations or large loops increase CPU usage, leading to a slower response. Handling many simultaneous requests can also result in thread contention or queueing delays.
- **Query Complexity:** Poorly optimized queries or missing indexes can slow down data retrieval and increase latency.
- **Database Locks:** Concurrent writes or updates on the same rows/tables can lea blocking or deadlocks.

- **Multiple Service Calls:** Multiple internal service calls (in a microservices architecture) can accumulate latencies. Also, if the system depends on external services, their response times can add up.

Throughput

Throughput refers to the number of transactions, requests, or operations a system processes within a given period (such as requests per second, database transactions per minute, etc).



For example, if an API processes 5,000 requests per second, that's its throughput ur current conditions.

Key metrics to measure throughput are as follows:

- **Requests per second** (**RPS**): The number of requests an API or server handles p second.
- **Transactions per second** (**TPS**): Common in databases and financial systems.
- **Queries per second** (**QPS**): Used to measure the database's ability to handle read/write queries.
- **Bandwidth utilization**: Measures how much network capacity is being used.

Throughput measures system capacity, not individual request speed.

Low response time does not always mean high throughput. If a system processes requests quickly but cannot handle many at once (due to CPU, memory, or database constraints), throughput will still be low. Typically, a high-throughput system is designed to handle many concurrent users while maintaining acceptable response times.

Some potential bottlenecks to throughput are as follows:

- **CPU and Memory Constraints:** If the server's CPU usage reaches 100%, it cani efficiently process additional requests. For example, a high-traffic e-commerce site may experience a surge in users, maxing out its servers and reducing throughput.
- **Database Performance**: Slow queries, excessive locking, or unoptimized indexe can limit database transaction throughput. For example, a system that handles payments experiences deadlocks when too many transactions try to update the same account balance.
- **I/O Bottlenecks** (**Disk and Network**)**:** Writing to disk can become a bottleneck the disk is slow.
- **Concurrency Limitations:** Thread pools, connection pools, or backend services may have limits that throttle the number of simultaneous requests.

- **Rate Limits and Throttling:** Many external services impose request limits to prevent abuse. For example, a weather API may allow only 1K requests per minute, capping how many users the app can serve simultaneously.

Resource Utilization

Resource utilization refers to how effectively a system employs its available hardwa and infrastructure, specifically CPU, memory, input/output (I/O), and network bandwidth.

When utilization levels are balanced, the system can perform tasks efficiently witho over-provisioning (wasting resources) or under-provisioning (causing bottlenecks). I cloud or virtualized environments, efficient utilization translates to lower operatior costs (where we pay for fewer or smaller instances) and better performance.

At the same time, it supports scalability because an optimally utilized system can expand or contract its resources without significant overhead.

Poor resource utilization can degrade system performance and availability in multi ways.

- For instance, if CPU usage is consistently at 100%, applications may slow to a crawl or become unresponsive.
- With insufficient memory, critical processes might be forced into swap space, leading to significant slowdowns or crashes.
- If I/O operations are slow (for example, due to a saturated disk or storage network), requests pile up, increasing response times.
- Likewise, network bandwidth constraints can choke data transfers, especially i microservices architectures or data-heavy applications, causing timeouts and failed requests.

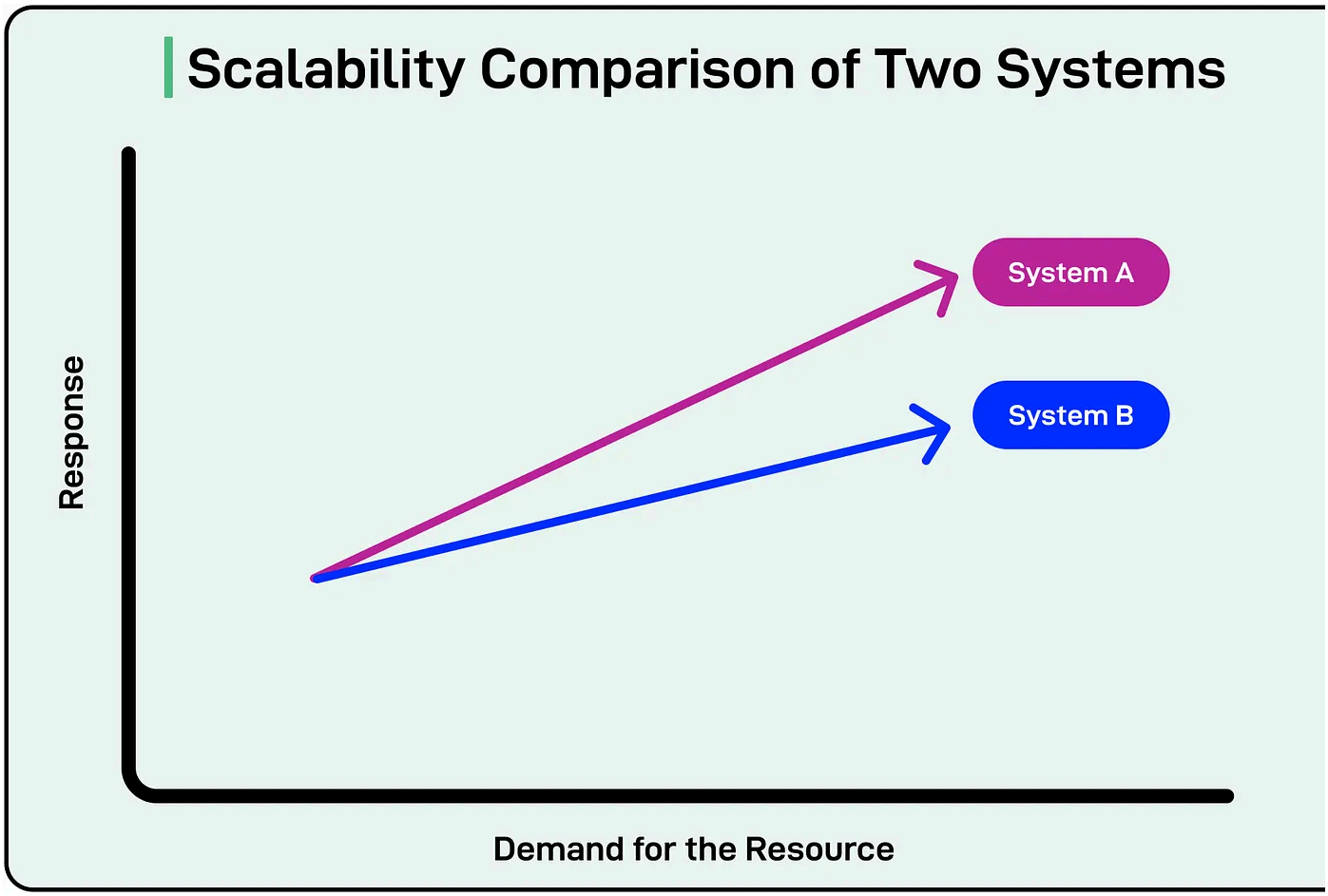
In all such scenarios, end users experience longer wait times, decreased reliability, a possibly complete service outages. This underscores why effective resource utilizati is so critical for modern software systems.

Scalability

Scalability is a system's ability to handle increasing workloads without sacrificing performance or reliability.

It ensures that as user demands grow, whether in the number of requests, data volu or transaction rates, the system can expand capacity in a controlled and cost-effecti manner.

The diagram below shows how the scalability of two systems can be compared.



There are two main types of scaling approaches available to developers:

- **Horizontal Scaling (Scale Out):** This involves adding more machines or nodes t distribute the workload across multiple instances. For example, using auto-scal groups in cloud environments and splitting large datasets across multiple databases (shards), each handling a portion of the overall load.
- **Vertical Scaling (Scale Up):** This involves increasing the power of an existing machine by adding more CPU, memory, or storage. For example, migrating fro small VM instance to a larger one with additional cores and RAM.

With horizontal scaling, we can continuously add capacity. This is often more fault-tolerant because the failure of one node doesn't bring down the entire system. However, it may introduce orchestration complexities and increased network overhead.

On the other hand, vertical scaling is straightforward to implement and existing applications may need minimal reconfiguration. However, it has an upper limit in terms of hardware constraints and can become expensive when major upgrades are required.

Capacity

Capacity refers to the maximum volume of work a system can process before its performance or reliability starts to decline. This could be measured using paramete like concurrent users, transactions, or data storage.

Once a system surpasses this limit, latency may spike, throughput may drop, and us can experience timeouts or errors.

This makes capacity planning a significant activity for the following reasons:

- **Prevent Service Outages:** A system that exceeds its capacity can crash or becor unusably slow, leading to lost revenue and damaged user trust.

- **Ensure Smooth User Experience:** Users expect consistent response times and reliability. Adequate capacity keeps the application performant under both nor and peak loads.
- **Optimizing Costs:** Over-provisioning wastes resources, while under-provisioni hurts performance. Capacity planning aims for a balance between performance needs and budget constraints.
- **Support Scalability:** By understanding capacity thresholds, teams can design scalable architectures using techniques like auto-scaling, clustering, or caching

But how can one estimate and plan for capacity?

A few tips are as follows:

- Examine typical usage, peak periods (for example., holiday sales), and business cycles (daily, weekly, and monthly). Use metrics like average requests per secon peak concurrent users, or daily transaction volume to determine typical loads.
- Simulate user traffic that steadily increases until the system shows signs of stre
- Look at past traffic spikes, sales periods, or seasonal fluctuations. Estimate futu demand by extrapolating growth rates and overlaying known business events su as new feature launches and marketing campaigns.

Availability

Availability measures how often a system is operational and accessible to users ove given period.

In practical terms, a highly available system has minimal downtime even under unexpected failures or maintenance windows. Moreover, such a system quickly recovers if an outage occurs. Reliability ties closely to availability, focusing on the system's ability to perform its intended functions without errors over time.

Increasing availability by even a fraction of a percent can significantly reduce downtime, but costs and complexity. Availability is often expressed as a percentage year, translating to a specific amount of downtime.

Common targets include:

- **99.9%** (“**Three Nines**”)
  - **Downtime:** ~8.76 hours of unavailability per year
  - **Usage:** Generally acceptable for many web applications or internal enterpri systems
- **99.99%** (“**Four Nines**”)
  - **Downtime:** ~52.6 minutes per year
  - **Usage:** Often targeted by mission-critical services, e-commerce platforms, financial applications
- **99.999%** (“**Five Nines**”)
  - **Downtime:** ~5.26 minutes per year
  - **Usage:** High-stakes scenarios, such as emergency response systems or large scale financial transaction processors.

The table below shows the various availability percentages and their downtime figures:

Availability

Availability %	Downtime Per Year	Commonly Referred
99%	3.65 days	Two nines
99.9%	8.76 hours	Three nines
99.99%	52.56 minutes	Four nines
99.999%	5.26 minutes	Five nines
99.9999%	31.5 seconds	Six nines

Apart from specific architectural patterns, improving availability depends a lot on monitoring and implementing auto-failover mechanisms. Some basic tips are as follows:

- Track CPU, memory, disk usage, and response times using tools like Promethe Grafana, or CloudWatch.
- Periodically verify service endpoints to detect early signs of failure. Kubernetes similar platforms can restart containers or shift workloads to healthy nodes wh something fails, minimizing downtime.
- Automated notifications (email, SMS, Slack) inform on-call teams about anoma or outages.
- Automatically route requests away from unhealthy nodes or data centers.

Resiliency and Fault Tolerance

Resiliency refers to a system's ability to recover after encountering an error or unexpected condition whether it is a server crash, network disruption, or data cente outage. A resilient system may temporarily degrade in performance or disable certa features when components fail, but it will eventually bounce back to normal operat

In contrast, fault tolerance focuses on the system's ability to function even as components fail, ideally without noticeable downtime or service degradation.

In other words, resiliency focuses on recovering after failures, while fault tolerance aims to continue operating seamlessly despite failures. Both concepts help ensure a robust, user-friendly experience, but take different approaches to dealing with failu

In distributed systems, failures are inevitable, whether it is because of a slow netwo link, a crashed process, or a corrupted data center. Adopting a mindset of "everythi fails, all the time" encourages developers to consider what happens when a service, database shard, or external API becomes unavailable. For instance:

- If a recommendation service goes down, the system might serve fallback suggestions or simply hide that section instead of blocking the entire user experience.
- When a primary database is unreachable, read from a replica or cached data sto to maintain partial functionality.
- Prevent the calling service from waiting indefinitely and triggering a chain reaction of failures.

Companies invest a lot in making their systems resilient and fault-tolerant.

For example, Netflix popularized resiliency engineering with a tool called Chaos Monkey, which randomly terminates instances within their production environmer By doing so, teams learn which services or components are not designed to handle unexpected shutdowns. Also, regular chaos experiments ensure new features and deployments remain resilient.

Disaster Recovery

Disaster recovery (DR) refers to the strategies and processes an organization uses to restore critical systems and data after a catastrophic event such as hardware failure, natural disasters, or cyberattacks. Its primary goal is to minimize downtime and data loss, ensuring business continuity.

Two key metrics drive DR planning:

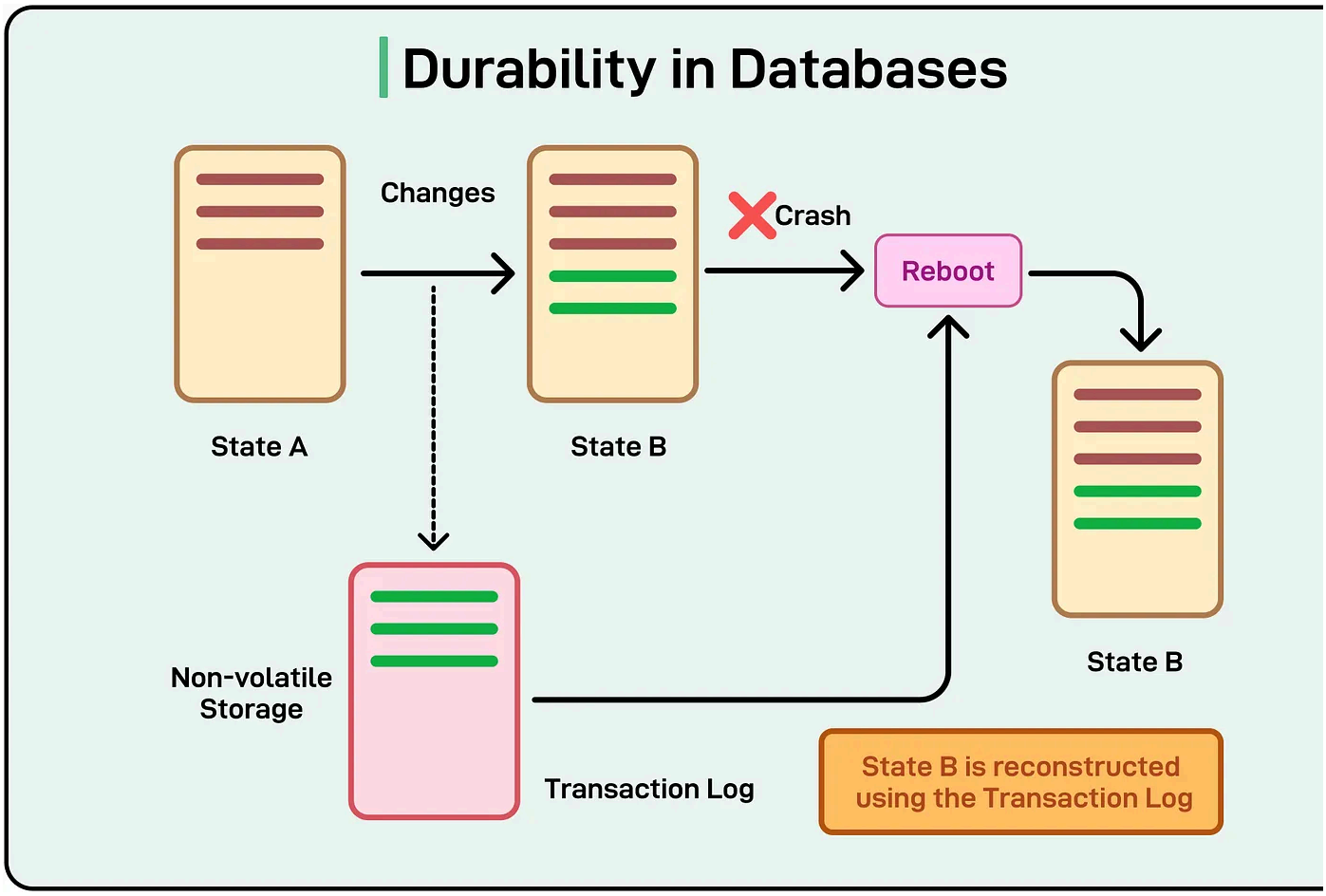
- **Recovery Time Objective (RTO):** The maximum acceptable time a system or service can be offline after a disaster. For example, if the RTO is 4 hours, the organization must fully restore operations within 4 hours of a major outage. A shorter RTO demands more sophisticated recovery solutions (such as automated failover), potentially increasing costs and complexity.
- **Recovery Point Objective (RPO):** The maximum acceptable amount of data loss measured in time. If the RPO is 1 hour, we must be able to recover all data up to at least 1 hour before the disaster occurs. Tighter RPOs require more frequent or real-time data replication, impacting storage, bandwidth, and infrastructure choices.

Durability

Durability refers to the guarantee that once data has been committed or saved, it will remain accessible and intact even if the system experiences crashes, power outages, or other failures.

In transactional systems (for example, databases), durability ensures that successfully completed transactions will not be "rolled back" or lost. In simpler terms, once the system confirms that the data is stored, it should be permanent and recoverable under normal failure conditions.

The diagram below shows the concept of durability in databases.



Some common strategies to ensure durability are as follows:

- **Replication:** Storing multiple copies of the same data across different servers o data centers. Even if one node or location fails, a replica can serve the latest da In a multi-master replication setup, each node keeps a near-real-time copy of t data, so no single point of failure exists.
- **Write-Ahead Logging:** Before changing the main data files, the system writes t intended changes to a log or journal. If the system fails mid-operation, it can u this log to “replay” or roll back changes to maintain consistency.
- **Snapshotting:** This involves periodically capturing the entire state of a databas or storage system at a point in time. It allows recovery to a known consistent st especially if corruption or accidental deletion is discovered later.

Consistency also plays a role in durability. In strongly consistent systems, every rea operation reflects the most recent write, ensuring data appears the same across all nodes at all times. This typically enhances perceived durability. Once data is writte it's quickly replicated and visible everywhere.

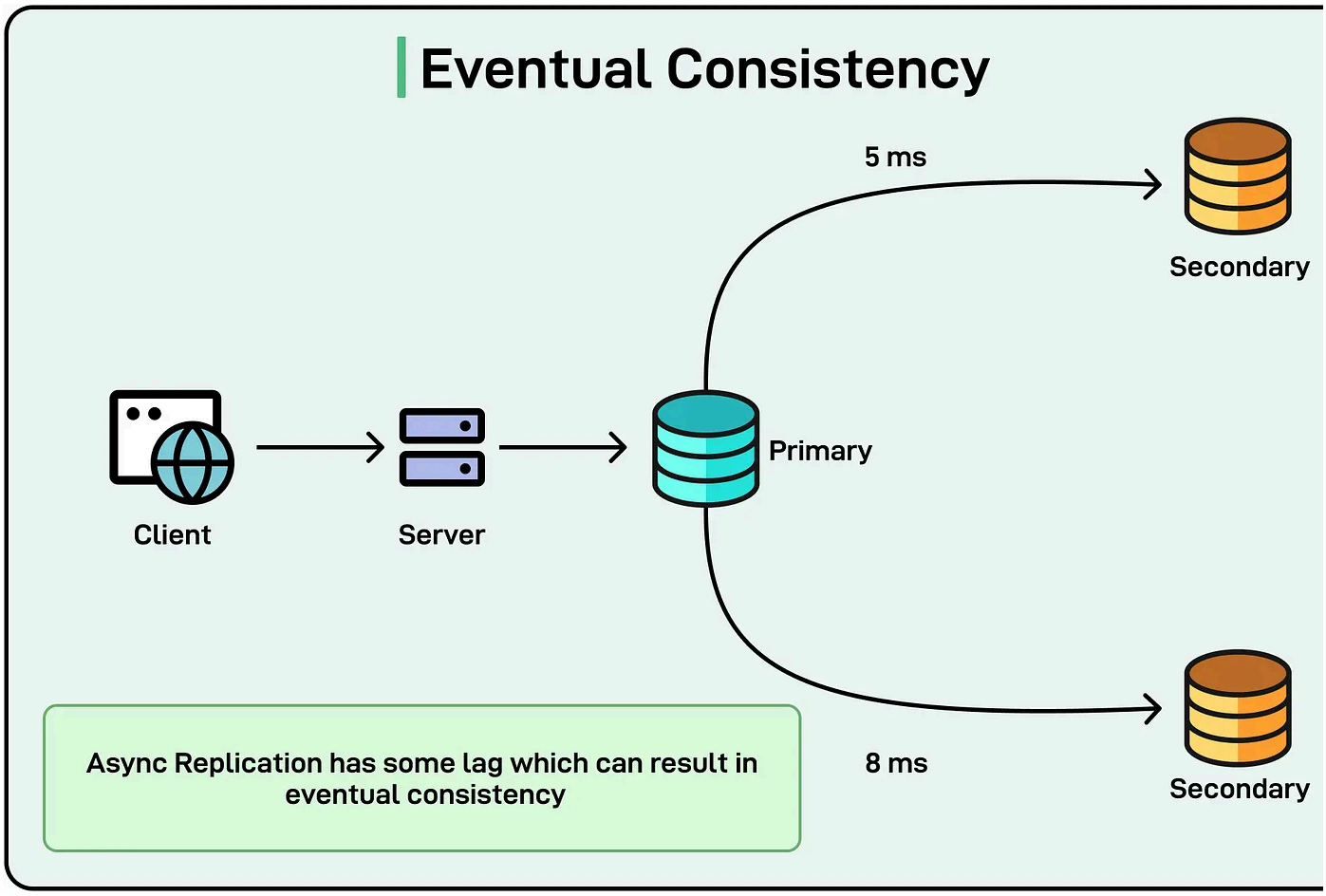
On the other hand, in eventual consistency scenarios, writes propagate over time. Different replicas might see different data until they converge, creating brief windc where data could be lost or overwritten if a node fails before synchronizing. In an eventually consistent system, there may be short-lived data "gaps" where not all replicas have the latest data. A crash during that gap can cause the newest data to b lost unless proper safeguards are in place.

Consistency

Consistency in distributed systems refers to how data is kept synchronized across multiple nodes or replicas, ensuring that reads reflect the most recent writes under specific rules.

There are two types of consistency models:

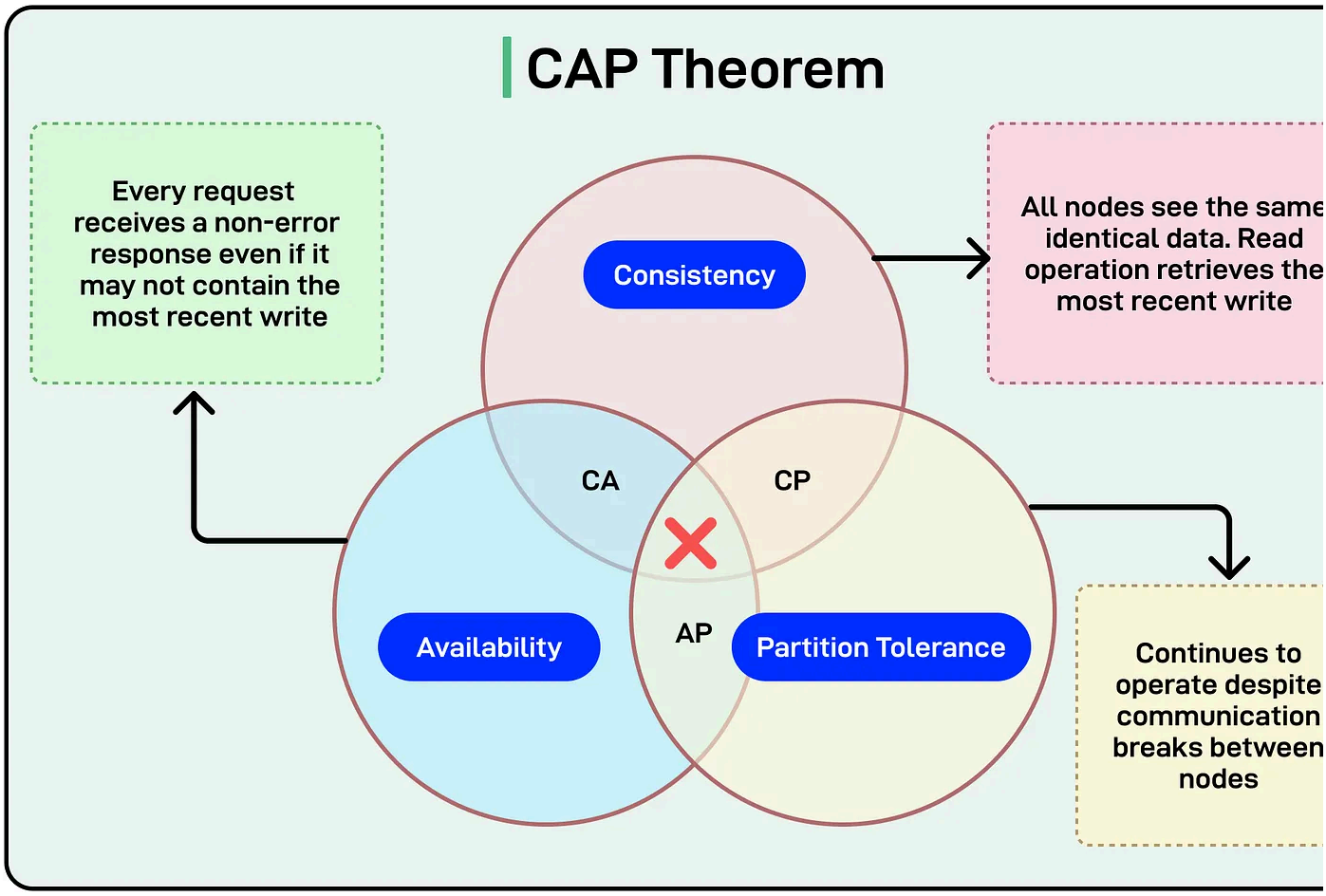
- **Strong Consistency:** After a write operation completes, any subsequent read operation (to any node in the system) will return the latest updated value. This approach is important for financial transactions, inventory management, or an domain requiring immediate, consistent data for correctness. Traditional relational databases often strive for strong consistency.
- **Eventual Consistency:** Updates propagate over time. Different replicas may temporarily see different states, but given enough time (assuming no new write all replicas converge to the same state. It is often used in social media feeds, content caches, or analytics platforms where stale data for a short period is acceptable in exchange for higher availability and scalability. Many NoSQL systems like Amazon DynamoDB, and he Cassandra adopt an eventually consistent model by default.



Depending on the application's needs (strict correctness or high scalability) develop can choose different consistency models.

Eric Brewer's CAP theorem states that in the presence of a network partition, a distributed system must choose between consistency and availability. It cannot fully guarantee both.

The diagram below shows a pictorial representation of the CAP theorem:



Since network partitions are inevitable in distributed systems, developers often nee to pick trade-offs such as:

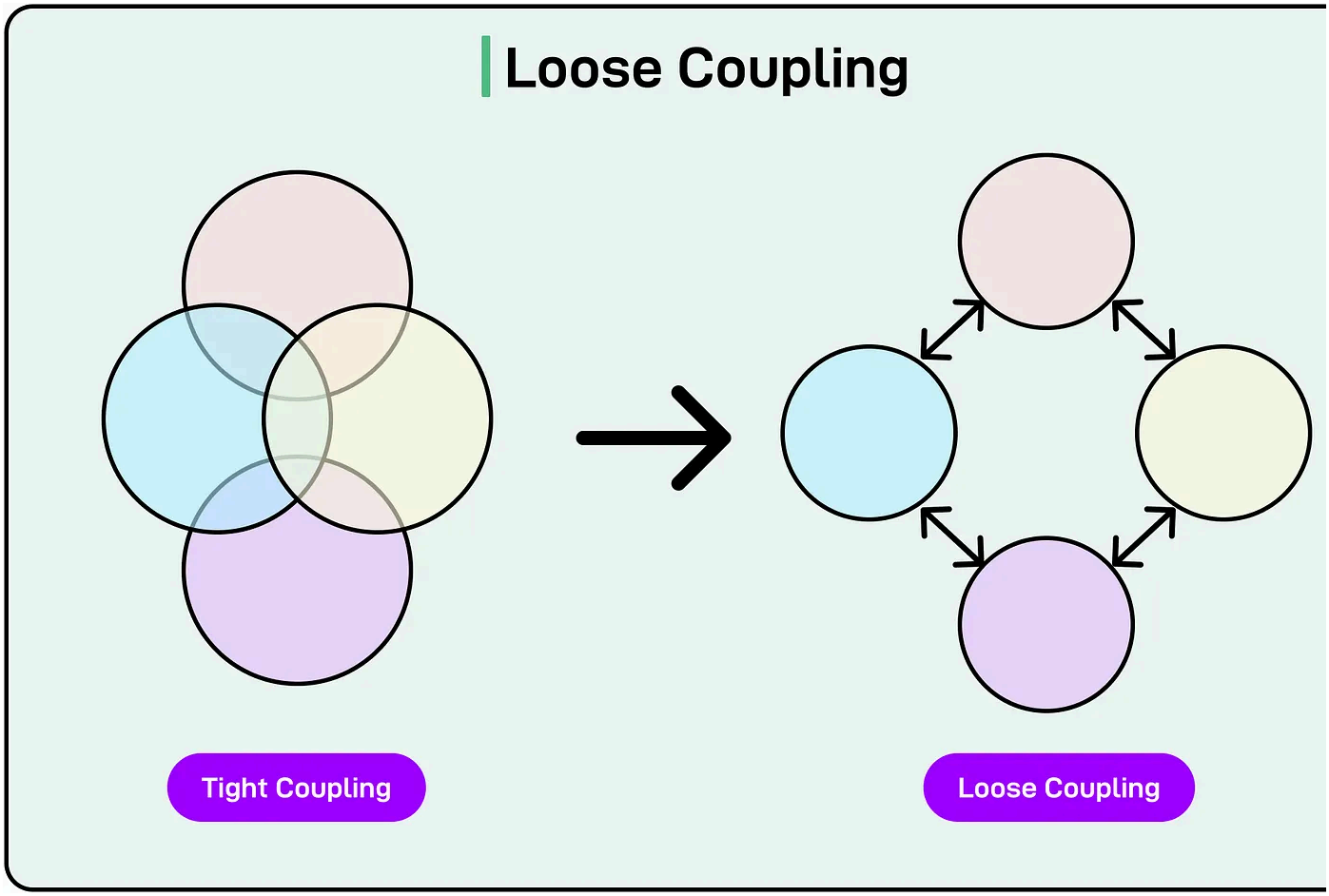
- **CP Systems:** Choose Consistency and Partition Tolerance over availability. The system may reject requests during a partition if it cannot maintain consistency.
- **AP Systems:** Choose Availability and Partition Tolerance over strict consistenc The system remains operational but may serve slightly stale reads during partitions.

Modularity

Modularity is a design principle that involves breaking a large codebase into smalle self-contained units (modules or components).

It benefits large codebases in multiple ways:

- **Easier Maintenance and Updates:** Each module handles a well-defined responsibility, reducing the chance of unexpected side effects when modifying particular feature. This means that a module can be upgraded, refactored, or replaced without having to change the entire system.
- **Reduced Coupling:** Minimizing the number of shared dependencies or global states prevents changes in one module from breaking another. Each module ca be unit-tested with minimal mocking of external components.



However, greater modularity can also have downsides:

- **Deployment Complexity**: Managing multiple modules or services often require more sophisticated CI/CD pipelines, container orchestration, and monitoring.
- **Network Latency**: In a service-based environment, inter-service calls can add round-trip overhead and potential points of failure.

- **Versioning and Compatibility**: Ensuring all modules stay compatible can be challenging, especially if they share interfaces or APIs.
- **Skill Requirements**: Developers, DevOps, and QA teams must understand distributed systems, orchestration tools, and advanced testing strategies.

Some common architectural styles that embrace the concept of modularity are as follows:

- **Microservices Architecture:** Each microservice is a standalone component, typically deployed and scaled independently.
- **Plugin-Based Systems:** The core application provides a framework for loading plugins, each delivering a discrete feature or extension.
- **Layered Architecture:** Divides the system into layers (for example, presentatio business logic, data access). Each layer is responsible for a specific aspect of th application

Testability

Testability refers to how easily and efficiently a software system's functionality and behavior can be validated through testing. The testing process can be manual or automatic.

A highly testable system typically has clear boundaries between components, minir dependencies, and well-defined interfaces, making it straightforward to isolate, observe, and verify individual parts. In other words, modularity is a desirable prope to improve the system's testability.

Design decisions can greatly affect testability.

For example, dependency injection (DI) is a great way to improve testability. Rather than creating dependencies internally, a class or module receives the objects it

depends on from an external source (such as a DI framework or a factory). This mak it easy to substitute real dependencies with mocks or stubs during testing.

Second, building the system with a clear separation of concerns enhances testabilit Independent components are simpler to unit-test, as each module performs a focus set of tasks and relies on well-defined interfaces.

Lastly, loose coupling and high cohesion improve testability. Components should h minimal knowledge of each other and handle closely related functionality within themselves. This decreases the risk that changes in one part of the system will brea tests elsewhere, making the system more predictable to test.

Some best practices for testing are as follows:

- **Unit Testing:** Focuses on verifying the smallest pieces of functionality in isolat (methods, classes, or modules). The idea is to keep tests fast and deterministic.
- **Integration Testing:** Ensures modules or services work together correctly. The goal is to test interactions between critical components in a test environment t mirrors production where possible.
- **System Testing** (**End-to-End**): This involves validating the entire application fl and user scenarios from start to finish. The idea is to test real user paths, including error handling and edge cases.

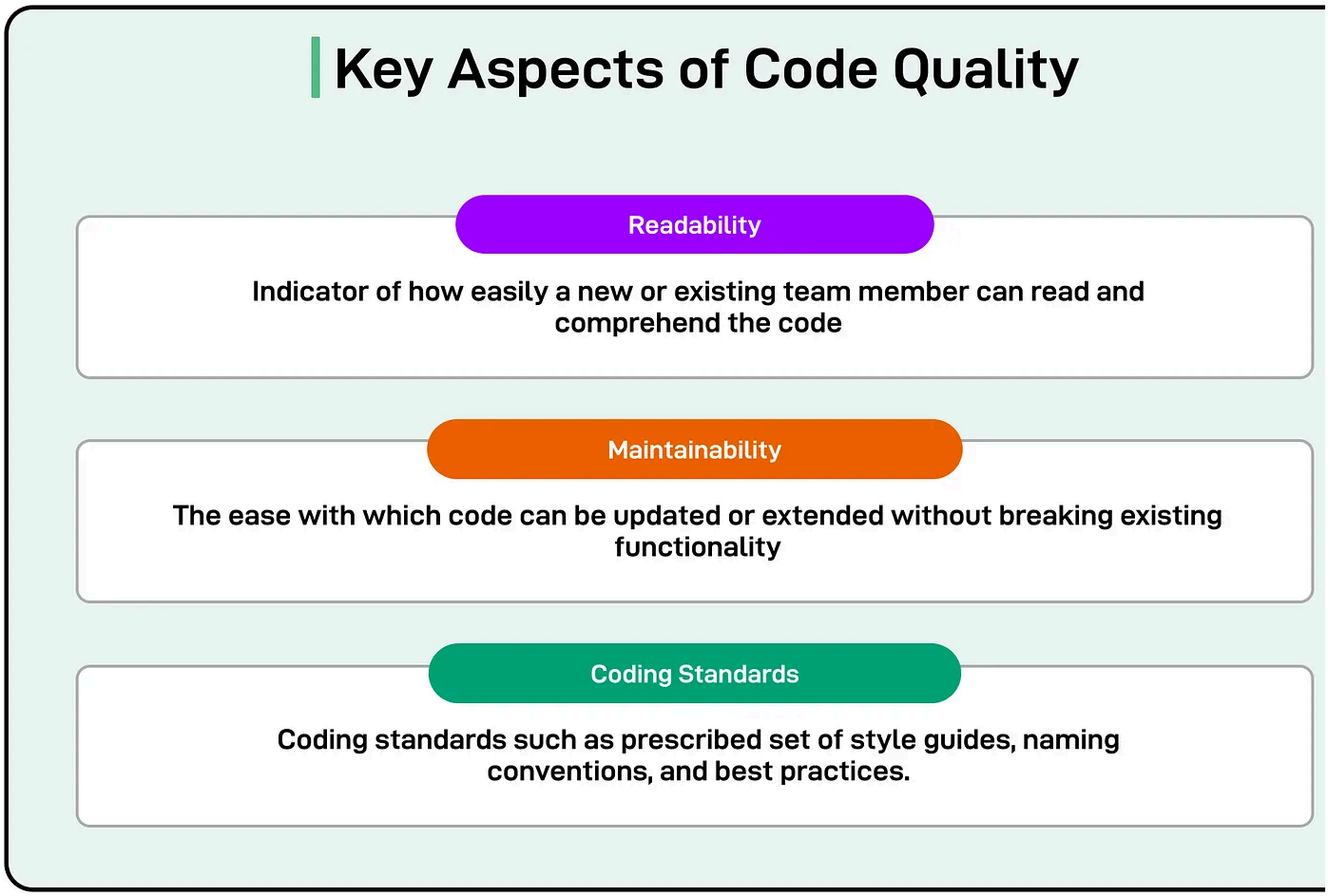
Code Quality

Code quality encompasses how well-written, understandable, and maintainable a software codebase is.

High-quality code generally has a clear structure, follows industry or organizationa standards, is free from obvious defects, and is easy to modify as requirements evolv Ensuring strong code quality pays dividends over a project's life, impacting performance, reliability, and overall development costs.

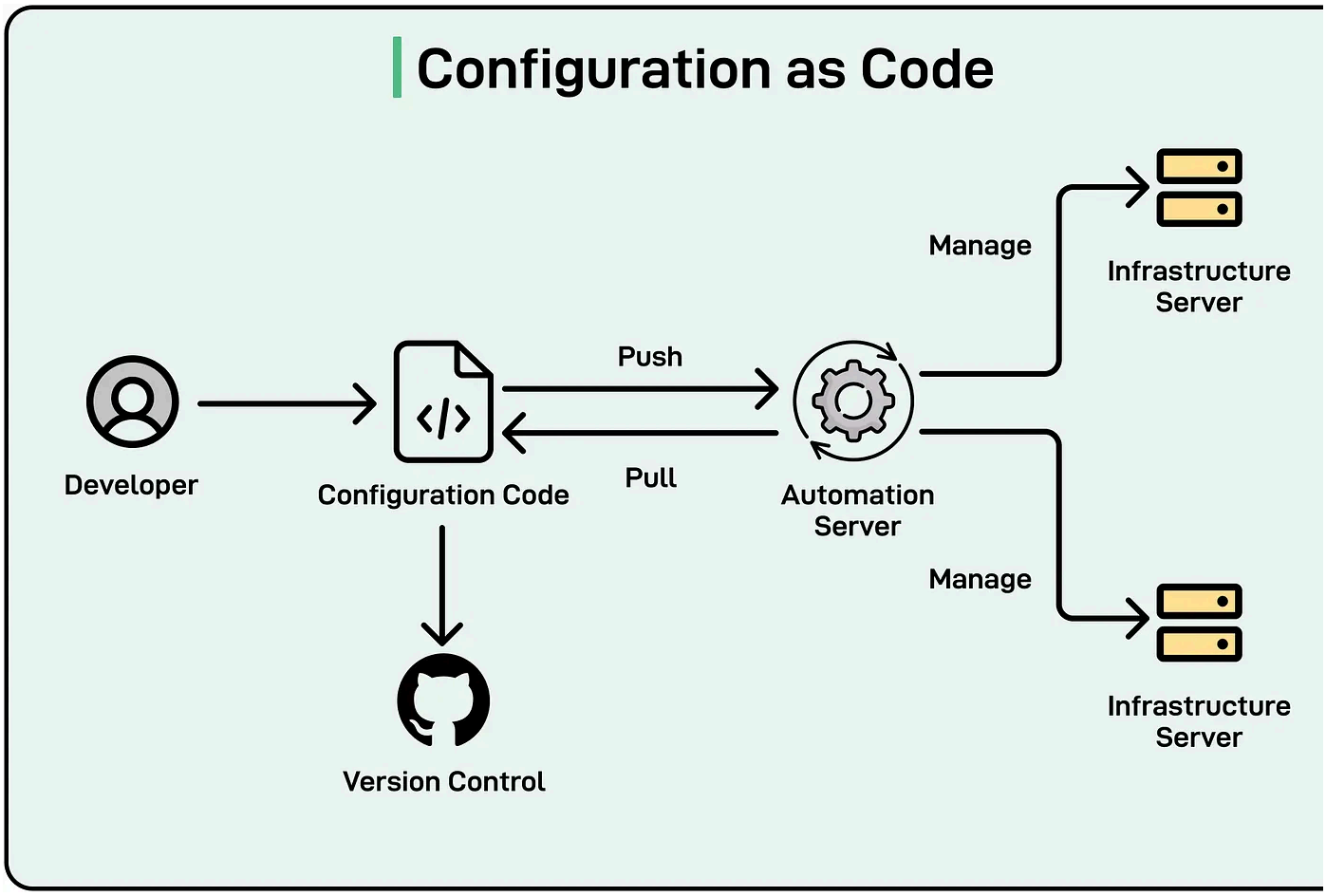
There are some key aspects of code quality that developers can keep in mind:

- **Readability:** This is an indicator of how easily a new or existing team member c read and comprehend the code. Readability can be improved by using meaning variable and function names (for example, calculateTotalPrice instead of calc). Other ways are keeping methods and classes concise to reduce cognitive load.
- **Maintainability:** This indicates the ease with which code can be updated or extended without breaking existing functionality. Developers should aim for a single responsibility within classes or modules and organize code into logical packages or folders.
- **Coding Standards:** Code quality also depends on following coding standards su as a prescribed set of style guides, naming conventions, and best practices. The uniformity in naming and formatting makes cross-team collaboration smoothe



Configurability

Configuration encompasses the various settings (for example, database connection strings, API keys, and feature flags) that determine how software behaves in differe environments such as development, testing, staging, or production.



By separating configuration from code, teams can change behavior without rebuild or redeploying the entire application.

Some benefits of keeping configuration separate are as follows:

- **Deployment Flexibility:** Storing environment-specific details in code forces a r release for each environment change (for example, switching from a test to a production database). Automated pipelines can reduce manual intervention by

injecting environment values (like API endpoints or logging levels) at deployme time.

- **Easier Maintenance:** Keeping settings in a centralized configuration store or fi helps teams quickly audit and modify them. Developers don't have to search through code for environment details, minimizing the chance of inadvertently pushing secrets or production credentials to version control.
- **Environment-Specific Settings:** Test environments might log extensively for debugging, while production logs are throttled to conserve resources. Certain features can be turned on or off depending on the environment or user segmen without altering the code.

Some examples of tools and frameworks that help improve a system's configurabilit are environment variables, configuration files, feature toggles, and external configuration management services such as Hashicorp Vault, Consul, or Kubernete ConfigMaps.

Security

Security as a non-functional requirement ensures that a system's data and operatio are adequately protected from unauthorized access, tampering, or disclosure.

Security isn't a one-time activity. It requires ongoing patching, monitoring, testing, and compliance reviews. It involves multiple facets ranging from authentication an authorization to data confidentiality, integrity, and auditability.

Some key security considerations to keep in mind are as follows:

- **Authentication:** Verifying the identity of a user or system (for example, username/password, multi-factor authentication, etc).
- **Authorization:** Controlling what authenticated users or systems can do (role-based or attribute-based access).

- **Confidentiality:** Ensuring that data is accessible only to authorized entities.
- **Integrity:** Protecting data from being tampered with or altered by unauthorized parties.
- **Auditing and Logging:** Recording security-relevant actions to detect or investigate unauthorized activity.

Security considerations often need to be balanced with performance and usability.

Encryption, multi-factor authentication, and high volumes of logging can add latency or resource utilization. Also, stringent password policies or complicated multi-factor flows can frustrate end-users.

Some industries (such as finance, and healthcare) prioritize strict security due to regulatory or reputational risks. Other sectors might lean towards ease of use, with the caveat that user data still needs robust protection.

Usability

Usability describes how effectively, efficiently, and satisfactorily users can interact with a software system to achieve their goals.

Though it doesn't outline specific functional behaviors (for example, "the application must allow the user to submit a form"), it significantly affects whether the user experience is pleasant, productive, and accessible. Hence, usability is classified as a non-functional requirement. It specifies how the system should behave from a user experience standpoint, rather than what features it provides.

Developers employ usability heuristics, user testing, and A/B experiments to improve usability. A usable product reduces support costs, increases user satisfaction, and builds positive brand perception.

Key factors to consider while understanding usability are as follows:

- **Intuitiveness:** This is the degree to which users can perform tasks without extensive instruction or trial-and-error. For example, a navigation menu labele clearly with standard terminology is easier to figure out than one with cryptic icons or jargon.
- **Accessibility:** This ensures that the software can be used by people with disabilities (visual, auditory, motor, or cognitive). For example, adhering to WC (Web Content Accessibility Guidelines) to provide text alternatives for images, enable keyboard navigation, and support screen readers.
- **Ease of Learning**: It is an indicator of how quickly new users can understand a become proficient in the system's workflows.

Interoperability

Interoperability is the capacity of a software system to exchange data or functionali with other systems, APIs, or services, regardless of underlying platforms or protoco

By adhering to common standards and designing for compatibility, developers ensu smoother integrations, minimize rework, and ultimately deliver a more cohesive us experience across different technologies.

Interoperability is important for modern systems due to the following reasons:

- **Smooth Data Exchange:** Systems that speak a common language (like RESTful JSON, SOAP XML) can share information without custom, error-prone transformations.
- **Reduced Integration Costs:** Following established standards (like OAuth for authentication and OpenAPI for service definitions) eliminates the need for custom solutions each time a new integration is required. Consistent interface streamline onboarding for new partners or third-party developers.
- **Future-Proofing:** Systems designed with forward and backward compatibility evolve without breaking existing integrations. This is especially critical for lon

lived platforms (such as government systems or large enterprises) where upgrad can be costly or disruptive.

Some techniques to improve interoperability are as follows:

- **Build APIs with Versioning:** Include a version number in the REST endpoint (f example., /v1/users, /v2/users) to roll out new features without breaking existing consumers.
- **Adopt Standard Protocols and Formats:** Using widely recognized approaches (example, JSON, XML, OAuth) makes integration approachable for external stakeholders.
- **Document Deprecation Policies:** Announce deprecation, offer a migration pat and set a reasonable timeline before final removal.
- **Validate and Test Integrations:** Maintain integration test environments and sandbox APIs so partners can test changes before hitting production.
- **Monitoring:** Track usage patterns to see if older endpoints remain heavily used that deprecation can be managed without disruption.

Summary

In this article, we've looked at various non-functional requirements in great detail a how we can measure them or improve upon them.

Some key learning points are as follows:

- Response time or latency is the interval between a user action (like clicking a button or making an API request) and the system's corresponding response (su as rendering the next page or returning data).
- Throughput represents how many operations a system handles per time unit. Bottlenecks (CPU, database, I/O) must be identified and optimized for high-volume performance.

- Efficient use of CPU, memory, I/O, and bandwidth drives cost savings and predictable performance, while poor utilization can lead to slowdowns or outag
- Horizontal scaling (scale out) adds more machines and vertical scaling (scale up adds more power. Both can meet growing demands but involve trade-offs in complexity and cost.
- Capacity is the maximum load a system can handle before performance degrad Planning for capacity ensures smooth user experiences during traffic spikes.
- Availability is often expressed as “nines” (99.9%, 99.99%, etc.) and dictates redundancy, monitoring, and failover strategies to minimize downtime.
- Resiliency and fault tolerance emphasize recovery and continued operation despite failures. Patterns like circuit breakers and bulkheads help systems degr gracefully instead of collapsing.
- Recoverability sets how fast a system must be restored.
- Durability ensures committed data remains persistent even after crashes or pov losses, relying on replication, journaling, and snapshotting.
- Strong consistency guarantees immediate alignment of data across nodes. On t other hand, eventual consistency prioritizes availability and scalability but acce temporary data discrepancies.
- Modularity is about breaking systems into self-contained components lowering complexity, enhancing maintainability, and facilitating independent testing an updates.
- Testability involves designing with loose coupling and dependency injection makes systems easier to test.
- Readable, standardized, and well-structured code reduces bugs, improves long-term performance, and prevents costly technical debt.
- Storing environment-specific settings externally (such as environment variable enables flexible, reliable deployments, and mitigates security risks.

- Security covers authentication, authorization, confidentiality, integrity, and auditing. A secure design balances performance, usability, and regulatory compliance.
- Usability addresses intuitiveness, accessibility, and ease of learning. Even smal improvements can greatly boost user satisfaction.
- Interoperability and compatibility ensure seamless communication and data exchange across systems or services, leveraging standards (REST, SOAP, OAuth and versioning to prevent breaking changes.

198 Likes · 11 Restacks

Discussion about this post

Comments Restacks

Write a comment...